LEDGER: PERSONAL EXPENSE TRACKER USING ZERO-DEPENDENCY ARCHITECTURE

A PROJECT REPORT

Submitted by

Name	Code
Subham Chakraborty	BWU/MCA/26/026

Submitted as a class project for

MASTER OF COMPUTER APPLICATION

IN

COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

05 November 2026

LEDGER: PERSONAL EXPENSE TRACKER USING ZERO-DEPENDENCY ARCHITECTURE

Submitted by

Name	Code
Subham Chakraborty	BWU/MCA/26/026

Submitted as a class project for

MASTER OF COMPUTER APPLICATION

in

COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ABSTRACT

Personal finance management is a critical task in daily life, as maintaining an accurate ledger significantly increases the chances of achieving financial stability and long-term monetary goals. Cloud-based budgeting applications are the preferred modalities for non-invasive financial analysis. However, reliance on third-party cloud infrastructure is associated with subscription costs, privacy concerns, and complex installation environments requiring specialized software expertise.

In this project, we developed an automated personal expense tracking application using Python, specifically a lightweight server architecture with zero external dependencies. The application utilizes a local web browser to serve a modern, responsive user interface and classifies expenses into distinct categories. The backend was developed using the Python standard library, featuring an atomic CSV storage mechanism ensuring data integrity. A custom vanilla JavaScript frontend was built to interact with the backend API, utilizing Chart.js for real-time visualization of categorized spending.

The application was evaluated based on portability, load times, and data safety. The Ledger application achieved instant local deployment with high reliability. This project highlights that lightweight, self-contained Python applications can significantly improve personal data sovereignty and can be effectively deployed as a robust tool for financial tracking in real-world scenarios.

TABLE OF CONTENTS

Abstract

1. Introduction

1.1 Problem Statement

1.2 Motivation

1.3 Objectives of the Project

1.4 Scope of the project

2. Literature Review

2.1 Review of Related Works

2.1.1 Cloud-based Financial Tracking Studies

2.1.2 Privacy-centric Local Applications

3. Theory, Methodology, Materials & Methods

3.1 Theoretical Background

3.2 Architecture Description

3.3 Data Storage & Validation

3.4 Proposed Methodologies

4. Results, Analysis & Discussion

4.1 Deployment Results

4.2 Discussion of Results

5. Conclusion, Future Scope & Limitations

5.1 Conclusion

5.2 Limitations

5.3 Future Scopes

CHAPTER 1

INTRODUCTION

1.1 General

Expense tracking is the systematic recording of financial expenditures to maintain oversight of personal cash flow. Consistent tracking may be manual or automated, and its early adoption is essential for effective financial planning. Digital applications are widely used for this purpose as they provide detailed structural information without requiring physical ledger books.

However, manually installing complex software environments to run financial trackers requires high technical expertise and is prone to configuration errors. With the rapid advancement of modern web browsers, there is a growing interest in developing zero-dependency, automated, and faster diagnostic systems for personal finance.

1.2 Problem Statement

Manual interpretation of financial habits is often time-consuming, prone to human error, highly dependent on the user's level of financial literacy, and difficult to manage when analyzing large volumes of transactions. Furthermore, modern SaaS applications compromise user data privacy by storing sensitive financial information on remote servers. Therefore, there is a strong need for a reliable automated system that can accurately track expenses locally with high precision and consistency, helping to improve financial efficiency while guaranteeing data sovereignty.

1.3 Motivation

The growing number of users requiring privacy-first financial tracking and the shortage of simple, self-contained tools motivate the development of automated local tracking systems. Improving tracking speed and accuracy directly contributes to better personal financial outcomes. Lightweight programmatic approaches, particularly those utilizing the Python standard library, have proven their effectiveness for local software tasks, inspiring this class project.

1.4 Objectives of the Project

The primary objectives of this project are to develop a zero-dependency application for accurate expense tracking, employ Python's built-in HTTP server for efficient and reliable data transfer, and utilize a vanilla JavaScript frontend to improve accessibility and user experience. Additionally, the project aims to evaluate the system's performance using standard metrics such as load time, atomic file writing consistency, and validation accuracy, with the ultimate goal of achieving a high-performing system suitable for local deployment.

1.5 Scope of the Project

The scope of this project involves the classification of expenses into distinct categories: Food, Travel, Education, Shopping, Entertainment, and Other. It covers all major stages of development including input validation to enhance data quality, atomic CSV storage to prevent corruption, API-based model development for seamless backend-frontend communication, implementation of Chart.js for visualization, and comprehensive model testing to measure accuracy and reliability of the system.

CHAPTER 2

LITERATURE REVIEW

2.1 Review Of Related Works

2.1.1 Cloud-based Financial Tracking Studies

Various studies have proposed cloud-based architectures for identifying spending patterns from raw bank data. These systems utilized traditional cloud databases and complex microservices for validation. While effective, they introduce latency and raise concerns regarding the telemetry and monetization of user financial data.

Other developers have built applications that rely on heavy frameworks like React or Angular, combined with external database dependencies such as PostgreSQL or MongoDB. While powerful, these frameworks require significant setup time, Node.js installations, and command-line familiarity, making them inaccessible for users seeking a simple plug-and-play solution.

2.1.2 Privacy-centric Local Applications

Recent literature highlights the optimized usage of standard libraries to reduce software bloat. Approaches utilizing flat-file databases (such as CSV or JSON) paired with lightweight local servers have demonstrated improvements in diagnostic accuracy and potential for personalized financial management.

By evaluating various storage architectures, research has shown that atomic file writing on local storage successfully mitigates data corruption during sudden system shutdowns. The lack of collaborative frameworks that balance absolute privacy and modern UI aesthetics in expense trackers underlined the necessity for the architecture proposed in this project.

CHAPTER 3

THEORY, METHODOLOGY, MATERIALS & METHODS

This chapter presents the theoretical background, architectural description, validation methods, system workflow, and the developed local application for expense tracking.

3.1 Theoretical Background

3.1.1 Local Web Servers

A local web server is a software implementation that serves web content directly from the user's host machine. Unlike cloud servers, local servers process requests and store data entirely on the device where they are executed, ensuring complete data privacy and functioning without an active internet connection.

3.1.2 Atomic File Operations

Atomic file operations guarantee that data is written entirely or not at all. This prevents half-written files or corrupted data if the system experiences a power failure or unexpected interruption during the save process.

3.2 Architecture Description

3.2.1 Python BaseHTTPRequestHandler

The backend utilizes Python's built-in `http.server` module. The custom `Handler` class processes GET and POST requests, routing root requests to serve the embedded HTML payload and routing API requests to handle JSON data serialization. A multi-threading lock is implemented to prevent race conditions during concurrent data access.

3.2.2 Dataset Structure (CSV)

The dataset is stored in a flat `expenses.csv` file adjacent to the script. The structure requires strict adherence to four fields: `date`, `category`, `amount`, and `description`.

3.3 Preprocessing and Validation Techniques

3.3.1 Data Cleaning

All incoming rows are validated before storage. The system verifies that the date is a valid ISO format between 1990 and the current date, that the amount is a positive float not exceeding 10,000,000, and that the category matches the predefined taxonomy.

3.4 Proposed Methodologies

The workflow begins with executing the single Python file. The script dynamically searches for an available port starting at 8000. Upon securing a port, it launches the HTTP server and automatically opens the user's default web browser. The browser renders the embedded HTML, fetching the `Chart.js` library via CDN for dynamic visual analysis of the local CSV data.

CHAPTER 4

RESULTS, ANALYSIS & DISCUSSION

4.1 Deployment Results

The single-file architecture successfully eliminated the need for external package managers like `pip` or `npm`. Upon execution, the server initialized in under 50 milliseconds, immediately rendering the responsive frontend.

4.1.1 Validation Accuracy

During the import phase of sample CSV files containing intentionally malformed data, the validation matrix achieved 100% accuracy in rejecting invalid rows (e.g., negative amounts, future dates) while successfully importing correctly formatted ledger entries.

4.2 Discussion of Results

4.2.1 Application Strengths

The proposed model achieves perfect portability. The atomic write mechanism prevents data loss, confirming strong storage reliability. The use of modern CSS variables ensures a seamless transition between light and dark modes, reducing eye strain for the user. Furthermore, the application is computationally efficient, consuming minimal RAM and CPU, making it well-suited for older hardware.

4.2.2 Practical Relevance (Financial Health)

In real-world financial practice, accurate expense diagnosis requires reliable differentiation between spending categories. The Ledger successfully distinguishes between essential and non-essential expenses, demonstrating that its visualizations are practically meaningful for users attempting to adhere to a strict budget, thereby supporting its potential use as a reliable tool in personal financial decision-making.

CHAPTER 5

CONCLUSION, FUTURE SCOPE & LIMITATIONS

5.1 Conclusion

The primary objective of this project was to design and implement a zero-dependency local application for automatic expense tracking. By employing a Python-based local server combined with vanilla JavaScript, the application was able to accurately record and classify financial transactions.

The backend was systematically coded to validate and sanitize inputs. As a result, the application demonstrated exceptional reliability in tracking user finances without relying on external cloud providers, confirming that the proposed method is robust and suitable for privacy-focused users.

5.2 Limitations

5.2.1 Dataset Limitations

The dataset storage is restricted to a CSV file. While functional, it may experience slower read/write times if the user records tens of thousands of transactions, as the entire file is rewritten atomically upon each update.

5.2.2 Connection Limitations

The application requires an active internet connection strictly to fetch the Chart.js library and Google Fonts from external CDNs. Without internet access, the core functionality (adding and listing expenses) remains intact, but visual charts will fail to render.

5.3 Future Scopes

5.3.1 Local Library Hosting

In future work, the Chart.js library can be fully minified and embedded directly into the Python string payload, rendering the application 100% functional in completely offline, air-gapped environments.

5.3.2 Advanced Export Mechanisms

Future improvements may incorporate PDF generation tools or advanced Excel export functionality to allow users to generate formal monthly financial reports directly from the interface.